

Chapter 12 .딥러닝_다중분류

- 다중분류 활성화 함수 및 손실함수
- 딥러닝 다중분류 실습

* keras 다중 분류

- softmax 출력단 활성화 함수 활용 (정규화된 지수 함수)

- 여러 개의 답 중 하나를 고르는 분류 문제
(꽃 품종 예측 분류, 생선 종류 예측 분류, 패션 잡화 종류 분류 등..)
- 둘 중에 하나를 고르는 이항 분류(binary classification)와 접근 방식이 조금 다름

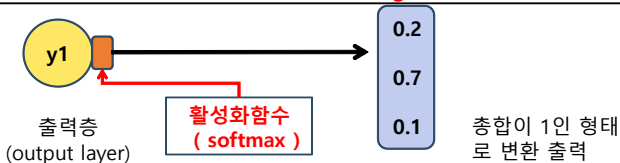
- 출력층 뉴런수

- 정답 개수 와 일치하는 출력 뉴런 개수 지정

- 타깃 데이터

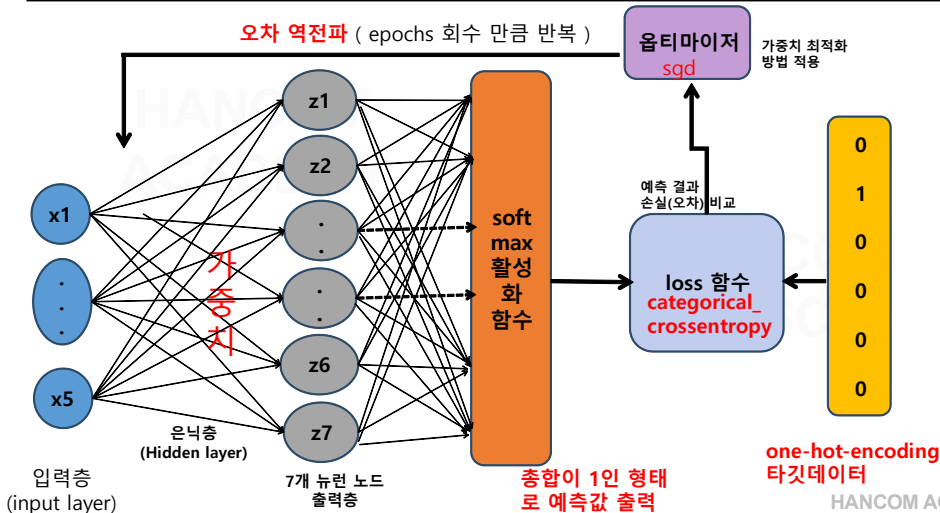
- 0 과 1 로 표현된 원-핫 인코딩(one-hot-encoding) 데이터 사용

* 다중 클래스 분류를 위해
예측값을 도출하는 가설함수로
softmax 함수 사용

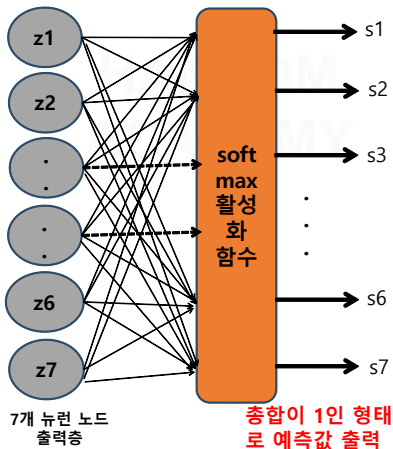


* keras _ 다중분류 모델 설계

오차 역전파 (epochs 회수 만큼 반복)



* softmax 활성화 함수 동작



[지수 함수 적용 $\text{np.exp}(x)$]

$$e_num = e^{z1} + e^{z2} + e^{z3} + e^{z4} + e^{z5} + e^{z6} + e^{z7}$$

$$s1 = \frac{e^{z1}}{e_num} \quad , \quad s2 = \frac{e^{z2}}{e_num} \quad \dots \quad s7 = \frac{e^{z7}}{e_num}$$

* 손실(비용) 함수 : 크로스 엔트로피 (cross entropy)

- : 서로 다른 두 확률 분포의 차이를 의미
- : 로지스틱 회귀 관점에서 **모델 예측값의 확률과 실제값 확률의 차이**
- : **예측값과 실제값의 차이를 가장 최소화(손실 최소화)하는 w를 구하는 방법**

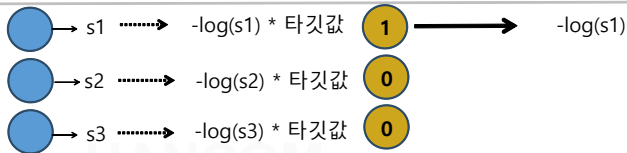
공식 : $-\sum_x p(x) \log q(x)$ $p(x)$: 실제 데이터의 분포, $q(x)$: 모델의 예측값의 분포

소프트맥스 다중 분류 : (다중분류, categorical cross entropy 손실함수)

$$\text{cost}(w) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k y_j^{(i)} \log(p_j^{(i)})$$

y_j : **실제값 원-핫벡터**, p_j : 예측값 확률
 k : 클래스 수

실제값이 원-핫벡터가 아닌 여러 정수형태일 경우
==> `sparse_categorical_crossentropy` 손실함수 사용



softmax 통과한 예측값

타겟(실제) 값

(실제값) $y_j = [0 \ 1 \ 0]$, (예측값) $p_j = [0 \ 1 \ 0]$ 일 경우

$$-\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} \log 0 \\ \log 1 \\ \log 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} -\infty \\ 0 \\ -\infty \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = 0$$

예측값과 실제값이
같아 손실이 0

(실제값) $y_j = [1 \ 0 \ 0]$, (예측값) $p_j = [0.7 \ 0.2 \ 0.1]$ 일 경우

$$-\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} \log(0.7) \\ \log(0.2) \\ \log(0.1) \end{bmatrix} = -\log(0.7) = 0.35$$

예측값과 실제값
차이로 손실이 0.35

예측값과 결과값이 비슷할수록 $\text{cost}(\text{loss})$ 는 줄어듦, 즉 실제값이 1인 위치의 예측값을 가능한 1로 가깝게 만들어주는게 목표!

cost 의 값을 줄여 나가는 방향으로 경사하강법 적용하여 최적화

* 케라스 딥러닝_생선종류_다중분류(softmax)

tensorflow Gpu 사용 메시지 미출력하게 하는 방법 : 아래 2줄 추가

```
import os  
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

```
import numpy as np  
import pandas as pd
```

과학적 표기법 대신 소수점 6자리까지 나타낸다.
np.set_printoptions(precision=6, suppress=True)

```
fish_data = pd.read_csv('fish_data.csv')
```

```
fish_input = fish_data[['Weight','Length','Diagonal','Height','Width']].to_numpy()  
print(fish_input[:5])
```

```
fish_target = fish_data['Species'].to_numpy()  
print(fish_data['Species'].unique())
```

```
print(fish_target)
```

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
Y_encoded = le.fit_transform(fish_target) # fish_target 문자열 데이터 ==> 숫자 인코딩 변환
```

```
print('Y_encoded : ', Y_encoded)
```

```
print('le.classes_ : ', le.classes_)  le.classes_ : ['Bream' 'Parkki' 'Perch' 'Pike' 'Roach' 'Smelt' 'Whitefish']
```

```
from tensorflow.keras.utils import to_categorical # 타깃 데이터 => 원핫인코딩 변환 (다중분류)
```

```
Y_onehot = to_categorical(Y_encoded)
```

```
print(Y_onehot)
```



```
# 훈련데이터 / 테스트데이터 셋 분리
```

```
from sklearn.model_selection import train_test_split
```

```
train_input, test_input, train_target, test_target = W
```

```
train_test_split(fish_input, Y_onehot, random_state=42)
```

```
print(train_input.shape)
```

```
print(test_input.shape)
```

```
#StandardScaler 클래스 활용 표준화 전처리
```

```
from sklearn.preprocessing import StandardScaler
```

```
Scaler = StandardScaler()
```

```
Scaler.fit(train_input)
```

```
train_scaled = Scaler.transform(train_input)
```

```
test_scaled = Scaler.transform(test_input)
```



```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
lr_model = Sequential()
lr_model.add(Dense(input_shape=(5,), units=7, activation='softmax'))
lr_model.compile(optimizer='sgd', loss='categorical_crossentropy',
                 metrics=['accuracy'])
```

```
lr_model.fit(train_scaled, train_target, epochs=1000, verbose=1)
```

```
score = lr_model.evaluate(test_scaled, test_target) # 모델 평가
print('Test acc : ', score[1])
```

Test acc : 0.8500000238418579

```
lr_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 7)	42

Total params: 42
Trainable params: 42
Non-trainable params: 0

```
le.classes_ : ['Bream' 'Parkki' 'Perch' 'Pike' 'Roach' 'Smelt' 'Whitefish']
```

```
print( lr_model.layers[0].weights )
```

```
print(test_target[0])
```

```
[0. 0. 1. 0. 0. 0. 0.]
```

```
print(le.classes_[np.argmax(test_target[0])])
```

```
Perch
```

```
pre = lr_model.predict(test_scaled[0:1])
```

```
print('pre : ', pre)
```

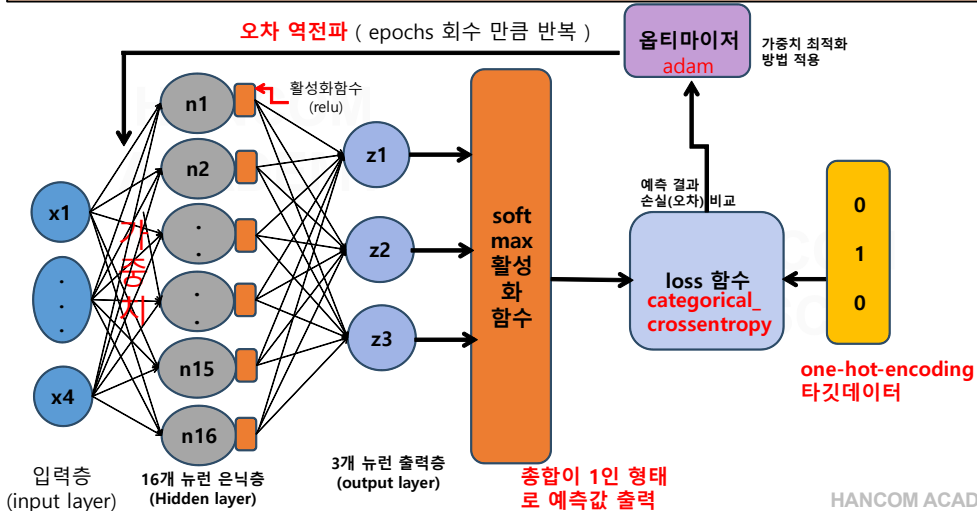
```
pre : [[0.00492999 0.10951679 0.46321955 0.01212831 0.25598413 0.14561695  
0.00860424]]
```

```
print(le.classes_[np.argmax(pre)])
```

```
Perch
```

* 케라스 딥러닝_아리스토텔_다중분류(softmax)

오차 역전파 (epochs 회수 만큼 반복)



```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

pd.set_option('display.max_columns',100)
pd.set_option('display.width',1000)
iris = load_iris()
#print(iris)
# sepal : 꽃받침, petal : 꽃잎
# 'data' array : [6.4, 3.1, 5.5, 1.8] sepal_length, sepal_width, petal_length, petal_width
# 'target_names': array(['setosa', 'versicolor', 'virginica'])
Iris_Data = pd.DataFrame(np.column_stack([iris['data'], iris['target']]), columns=['sepal_len','sepal_wd',
                                         'petal_len','petal_wd','target'])

print(Iris_Data.info())
#
#훈련할 샘플 데이터 추출
X_data = Iris_Data[['sepal_len','sepal_wd','petal_len','petal_wd']]
print(X_data.head())
# 레이블(클래스) 추출
Y_target = Iris_Data['target']
#print(Y_target.sample(10)) # target 값은 0, 1, 2 로 정수 인코딩 되어 있음.
```

#정수인코딩된 타깃레이블 원-핫-인코딩 변환

```
from tensorflow.keras.utils import to_categorical  
Y_onehot = to_categorical(Y_target.ravel())  
print(Y_onehot)
```

모델 설계

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense
```

```
model = Sequential()  
model.add(Dense(16, input_dim=4, activation='relu'))  
model.add(Dense(3, activation='softmax'))
```

#모델 컴파일

```
model.compile(loss='categorical_crossentropy', optimizer='adam',  
              metrics=['accuracy'])
```



다중 분류를 위한
손실함수 설정 (categorical_crossentropy)

#모델 훈련

```
model.fit(X_data, Y_onehot, epochs=50, batch_size=1)
```

모델 평가

```
print('accuracy : %.4f' %model.evaluate(X_data, Y_onehot)[1])
```

accuracy : 0.9733
약 97% 정확도로 분류

과학적 표기법 대신 소수점 6자리까지 나타냄

예측

```
np.set_printoptions(precision=6, suppress=True)
```

```
pre = model.predict(X_data[0:1])
```

```
print('pre : ', pre)
```

```
print('target_onehot : ', Y_onehot[0])
```

pre : [[**0.99908** 0.00092 0.]]
target_onehot : [1. 0. 0.]

```
# 'target_names': array(['setosa', 'versicolor', 'virginica'])
```

Thank You